

GOVERNMENT ARTS AND SCIENCE COLLEGE, ALANGUDI

College Code: BDU752

DEPARTMENT OF COMPUTER SCIENCE

PROJECT REPORT POCKETSMART AI

Your Smart Budget & Recommendation Assistant

SUBMITTED BY

TEAM I

Name	Email ID	Register Number
Vigneswaran K (Team Leader)	vickysuresh286@gmail.com	25057521802111034
Vetrivendhan M	vetrivendhan1205@gmail.com	25057521802111033
Kaleeshwaran S	skaleeshwaran6@gmail.com	25057521802111014
Gobi R	gopijuno2007@gmail.com	25057521802111009
Sudhagar M	msuthakar275@gmail.com	25057521802111029

Academic Year 2026–2027

PocketSmart AI

Your Smart Budget & Recommendation Assistant

Complete Project Development Report

Project Abstract

PocketSmart AI is a web-based smart budgeting and recommendation assistant designed to help users understand their income, expenses, available budget, savings, and purchasing options. The system accepts basic financial inputs, calculates spending and remaining budget, and provides recommendations according to the user's budget and selected category. The project is designed as a practical college-level software project using Python, Flask, HTML, CSS, JavaScript, and JSON/SQLite. The architecture can later be extended with authentication, charts, live product APIs, and an AI service.

Project Objectives

- Create a simple and user-friendly personal budget assistant.
- Calculate total expenses, remaining balance, and estimated savings rate.
- Organize spending and recommendations by category.
- Suggest affordable products or choices based on a defined budget.
- Provide a responsive interface suitable for desktop and mobile use.
- Build a project structure that can be maintained and uploaded to GitHub.

1. Brainstorming & Ideation Phase

Purpose

The brainstorming phase identifies the real-world problem, target users, possible features, and the overall idea for PocketSmart AI. The main problem identified is that many students and young users find it difficult to track spending and decide what purchases are affordable.

Problem Identification

- Users may not know how much money is available after regular expenses.
- Manual expense calculations can be time-consuming.
- Users may purchase products without comparing the purchase price with their available budget.
- Existing financial applications can be complicated for beginners.
- A simple assistant can combine budget calculation with basic recommendations.

Proposed Solution

PocketSmart AI combines a budget calculator and recommendation assistant in one simple application. A user enters income and expenses, views the calculated financial summary, selects a category and budget, and receives matching recommendations.

Key Features Identified

- Income entry
- Expense entry and categorization
- Budget calculation
- Savings estimation
- Category-based recommendations
- Simple dashboard
- Responsive user interface
- Future AI integration

2. Requirement Analysis Phase

Functional Requirements

ID	Requirement
FR01	The system shall accept monthly income.
FR02	The system shall accept multiple expenses and amounts.
FR03	The system shall calculate total expenses.
FR04	The system shall calculate the remaining budget.
FR05	The system shall estimate the savings percentage.
FR06	The system shall accept a recommendation category and budget.
FR07	The system shall return affordable matching recommendations.

Non-Functional Requirements

- Usability: The interface should be easy for beginners.
- Performance: Basic calculations should return quickly.

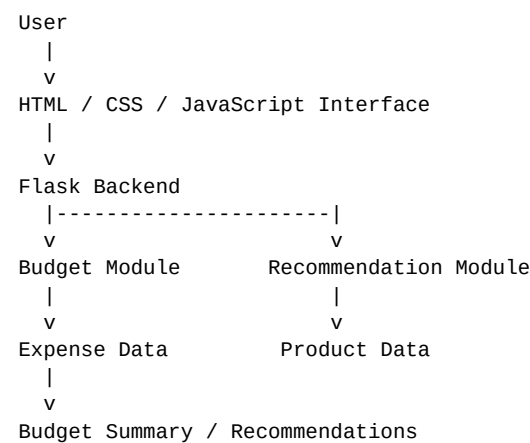
- Reliability: Invalid or missing numerical input should be handled safely.
- Maintainability: Backend, frontend, and data logic should be separated.
- Scalability: The design should allow future database and AI integration.
- Security: User input should be validated and sensitive data should not be exposed.

Hardware & Software Requirements

Minimum hardware: a computer with 4 GB RAM, modern browser, and internet access for development/deployment. Software: Python 3.10+, Flask, code editor such as VS Code, Git, GitHub account, and a modern web browser.

3. Project Design Phase

System Architecture



Main Modules

Module	Purpose
User Interface	Collect inputs and display results.
Budget Manager	Calculate expenses, balance and savings.
Recommendation Engine	Filter affordable items by category.
Data Layer	Store product and optional user data.
API Layer	Connect frontend requests with backend logic.

Basic Data Flow

- User enters monthly income.
- User enters expense categories and amounts.
- Frontend sends the data to the Flask API.
- Budget module calculates total expenses and remaining amount.
- Recommendation request is sent with category and available budget.
- Recommendation module filters products whose prices are within the budget.
- Results are displayed on the dashboard.

4. Project Planning Phase

Development Plan

Stage	Activities	Expected Output
Week 1	Ideation and requirement gathering	Problem definition and requirements
Week 2	System and UI design	Architecture and screen design
Week 3	Backend development	Flask APIs and budget module
Week 4	Frontend development	HTML/CSS/JS interface
Week 5	Recommendation module	Category-based recommendation engine
Week 6	Integration and testing	Integrated working application
Week 7	Documentation and demonstration	Final report and demo

Project Repository Structure

```
PocketSmart-AI/  
■■■ README.md  
■■■ requirements.txt  
■■■ .gitignore  
■■■ app.py  
■■■ src/  
■   ■■■ budget.py  
■   ■■■ recommendations.py  
■   ■■■ utils.py  
■■■ data/  
■   ■■■ products.json  
■■■ templates/  
■   ■■■ index.html  
■■■ static/  
■   ■■■ style.css  
■   ■■■ script.js  
■■■ screenshots/
```

5. Project Development Phase

Backend Development

```
from flask import Flask, render_template, request, jsonify

app = Flask(__name__)

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/api/budget", methods=["POST"])
def budget():
    data = request.get_json()
    income = float(data.get("income", 0))
    expenses = data.get("expenses", [])
    total = sum(float(x.get("amount", 0)) for x in expenses)
    remaining = income - total
    rate = (remaining / income * 100) if income else 0

    return jsonify({
        "income": income,
        "total_expenses": total,
        "remaining": remaining,
        "savings_rate": round(rate, 2)
    })

if __name__ == "__main__":
    app.run(debug=True)
```

Recommendation Logic

```
PRODUCTS = [
    {"name": "Wireless Earbuds", "category": "electronics", "price": 1499},
    {"name": "Laptop Stand", "category": "electronics", "price": 999},
    {"name": "Backpack", "category": "college", "price": 1299},
    {"name": "Water Bottle", "category": "fitness", "price": 699}
]

def recommend(budget, category):
    return [
        p for p in PRODUCTS
        if p["price"] <= budget
        and p["category"].lower() == category.lower()
    ]
```

Frontend Development

The frontend contains a dashboard with income and expense inputs, a calculate button, summary cards, category selection, and recommendation results. CSS provides responsive layout and JavaScript communicates with Flask using fetch() API calls.

GitHub Development Workflow

```
git init
git add .
git commit -m "Initial PocketSmart AI project"
git branch -M main
git remote add origin YOUR_GITHUB_REPOSITORY_URL
git push -u origin main
```

6. Project Testing Phase

Testing Objective

Testing verifies that calculations, input handling, API communication, recommendations, and user-interface functions behave correctly under normal and invalid conditions.

Test Case	Input / Action	Expected Result
TC01	Income = 30000, expenses = 10000	Remaining budget = 20000
TC02	Income = 0	No division error; savings rate handled safely
TC03	Expense amount is missing	System treats invalid/missing value safely
TC04	Budget = 1000, electronics	Only electronics priced <= 1000 are returned
TC05	Category has no matches	Empty recommendation result shown
TC06	Negative or invalid input	Validation/error message displayed
TC07	Mobile browser	Interface remains readable and usable
TC08	API request	Correct JSON response returned

Testing Types

- Unit Testing – individual budget and recommendation functions.
- Integration Testing – frontend, API, and backend communication.
- Functional Testing – verify each application feature.
- Usability Testing – check clarity and ease of navigation.
- Responsive Testing – test different screen sizes.
- Error Handling Testing – verify invalid input behavior.

7. Project Documentation Phase

Documentation Contents

- Project title and abstract
- Problem statement and objectives
- Requirement analysis
- System architecture and module descriptions
- Technology stack
- Development steps
- Testing methodology and test cases
- Screenshots of the application
- GitHub repository information
- Future enhancements and conclusion

Suggested README.md Sections

```
# PocketSmart AI

## About
Smart Budget & Recommendation Assistant.

## Features
- Budget calculation
- Expense tracking
- Savings estimation
- Category-based recommendations

## Tech Stack
Python, Flask, HTML, CSS, JavaScript

## Installation
pip install -r requirements.txt

## Run
python app.py

## Future Scope
AI API, database, authentication, charts, live product data.
```

Documentation Quality Checklist

- Use clear headings and consistent terminology.
- Include screenshots with short explanations.
- Document installation and execution steps.
- Explain the purpose of each major module.
- Record test cases and results.
- Keep GitHub README synchronized with the final project.

8. Project Demonstration Phase

Demonstration Flow

Step	Demonstration
1	Open the PocketSmart AI home page.

2	Enter a sample monthly income.
3	Enter several expenses and categories.
4	Click Calculate Budget.
5	Show total expenses, remaining budget, and savings rate.
6	Select a recommendation category.
7	Enter or use the available purchase budget.
8	Display matching affordable recommendations.
9	Show GitHub repository structure and source code.
10	Explain testing results and future enhancements.

Sample Demonstration

Example: A user enters an income of ■30,000 and expenses totaling ■18,000. The application displays ■12,000 as the remaining budget. The user then selects the electronics category with a ■1,500 recommendation budget. The system displays matching products priced at or below that amount. This demonstrates the connection between budget calculation and recommendation logic.

Expected Final Outcome

At the end of the demonstration, the evaluator should be able to see a working web application, understand the problem it solves, observe the main modules, verify the calculations, view recommendations, and inspect the GitHub-ready source structure.

Conclusion

PocketSmart AI demonstrates how a simple financial utility can be developed through a structured software development process. The eight phases—from brainstorming and requirement analysis through development, testing, documentation, and demonstration—provide a clear project workflow. The current design is suitable for a college project and provides a foundation for future improvements such as database storage, user authentication, spending charts, real-time product information, and advanced AI-powered recommendations.

Future Scope

- AI-powered natural-language financial assistance.
- SQLite/MySQL database integration.
- User registration and login.
- Monthly spending graphs and reports.
- Budget alerts and savings goals.
- Live product-price comparison.
- PDF/Excel report export.
- Cloud deployment and mobile/PWA support.